Lista de Exercícios: Teste Caixa-Branca
Teste de Mutação

Prof. André Takeshi Endo

(Exercício 1) Considere as quatro classes a seguir. Observe que existe um programa original e três mutantes; as mutações aplicadas em cada mutante estão destacadas em negrito.

Original.java

```
public class Original {
    //método retorna o maior elemento do vetor
   public int getMaior(int vetor[]) {
      int maior = vetor[0];
      for (int i = 1; i < vetor.length; i++) {
         if(vetor[i] > maior)
            maior = vetor[i];
      }
      return maior;
   }
}
```

Mutante1.java

```
public class Mutante1 {
    //método retorna o maior elemento do vetor
   public int getMaior(int vetor[]) {
      int maior = vetor[0];
      for (int i = 1; i < vetor.length; i++) {
         if(vetor[i] != maior)
            maior = vetor[i];
      }
      return maior;
   }
}
```

Mutante2.java

```
public class Mutante2 {
    //método retorna o maior elemento do vetor
   public int getMaior(int vetor[]) {
      int maior = vetor[0];
      for (int i = 2; i < vetor.length; i++) {
         if(vetor[i] > maior)
            maior = vetor[i];
      }
      return maior;
   }
}
```

Mutante3.java

```
public class Mutante3 {
    //método retorna o maior elemento do vetor
   public int getMaior(int vetor[]) {
      int maior = vetor[0];
      for (int i = 0; i < vetor.length; i++) {
         if(vetor[i] > maior)
            maior = vetor[i];
      }
      return maior;
   }
}
```

Implemente casos de teste em JUnit para matar cada um dos três mutantes; se algum dos mutantes for equivalente, justifique.

(Exercício 2) Considere as quatro classes a seguir. Observe que existe um programa original e três mutantes; as mutações aplicadas em cada mutante estão destacadas em negrito.

Original.java

```
public class Original {
   public int contarA(String word) {
      int contador = 0;

      for(int i = 0; i < word.length(); i++) {
         if(word.charAt(i) == 'a' ||
                              word.charAt(i) ==
'A')
            contador++;
      }
      return contador;
   }
}
```

Mutante1.java

```
public class Mutante1 {
   public int contarA(String word) {
      int contador = 0;

      for(int i = 0; i < word.length(); i++) {
         if(word.charAt(i) == 'a' ||
                              word.charAt(i) ==
'a')
            contador++;
      }
      return contador;
   }
}
```

Mutante2.java

```
public class Mutante2 {
   public int contarA(String word) {
      int contador = 0;

      for(int i = 0; i < word.length(); i++) {
         if(word.charAt(i) == 'a' ||
                              word.charAt(i) ==
'A')
            contador = contador + 1;
      }
      return contador;
   }
}
```

Mutante3.java

```
public class Mutante3 {
   public int contarA(String word) {
      int contador = 0;

      for(int i = 0; i < word.length()-1; i++)
{
         if(word.charAt(i) == 'a' ||
                              word.charAt(i) ==
'A')
            contador++;
      }
      return contador;
   }
}
```

Implemente casos de teste em JUnit para matar cada um dos três mutantes; se algum dos mutantes for equivalente, justifique.

(Exercício 3) Considere as quatro classes a seguir. Observe que existe um programa original e três mutantes; as mutações aplicadas em cada mutante estão destacadas em negrito.

Original.java

```
public class Original {
    public int calcularAnosBissexto(int ano[]) {
        int c = 0;
        for (int i = 0; i < ano.length; i++) {
            if(ano[i] % 400 == 0)
                c++;
            else if(ano[i] % 4 == 0 && ano[i] % 100 != 0)
                c++;
        }
        return c;
    }
}
```

Mutante1.java

```
public class Mutante1 {
    public int calcularAnosBissexto(int ano[]) {
        int c = 0;
        for (int i = 0; i < ano.length; i++) {
            if(ano[i] % 400 == 0)
                c++;
            else if(ano[i] % 4 == 0 && ano[i] % 100 != 0)
                i++;
        }
        return c;
    }
}
```

Mutante2.java

```
public class Mutante2 {
    public int calcularAnosBissexto(int ano[]) {
        int c = 0;
        for (int i = 0; i < ano.length; i++) {
            if(ano[i] % 400 == 0)
                c++;
            else if(ano[i] % 4 == 0 || ano[i] % 100 != 0)
                c++;
        }
        return c;
    }
}
```

Mutante3.java

```
public class Mutante3 {
    public int calcularAnosBissexto(int ano[]) {
        int c = 0;
        for (int i = 0; i < ano.length; i++) {
            if(ano[i] % 40 == 0)
                c++;
            else if(ano[i] % 4 == 0 && ano[i] % 100 != 0)
                c++;
        }
        return c;
    }
}
```

Implemente casos de teste em JUnit para matar cada um dos três mutantes; se algum dos mutantes for equivalente, justifique.

(Exercício 4) Considere as cinco classes a seguir. Observe o programa original no lado esquerdo e três mutantes no lado direito. As mutações aplicadas em cada mutante estão destacadas em negrito e identificadas em comentários. Perceba que some as linhas mutadas são apresentadas nos mutantes. O demais trechos de código são iguais ao programa original.

Pessoa.java

```
public class Pessoa {
   private String nome;
   private int idade;

   public Pessoa(String pNome, int pIdade) {
      nome = pNome;
      idade = pIdade;
   }

   public String getNome() { return nome; }

   public int getIdade() { return idade; }
}
```

Original.java

```
public class Original {
   public String definirFaixaEtaria(Pessoa p)
                                 throws RuntimeException {
      if(p.getIdade() < 0 || p.getIdade() >= 110)
          throw new
               IllegalArgumentException("idade invalida");

      int idade = p.getIdade();
      String tipo = "";
      if(idade <= 11)
         tipo = "crianca";
      else if(idade <= 18)
         tipo = "adolescente";
      else if(idade <= 59)
         tipo = "adulto";
      else
         tipo = "idoso";

      return p.getNome() + " eh " + tipo;
   }
}
```

Mutante1.java

```
public class Mutante1 {
   public String definirFaixaEtaria(Pessoa p)
                                 throws RuntimeException {
      if(p.getIdade() < 0 && p.getIdade() >= 110)  // '||' -> '&&'

      //O restante igual ao original
   }
}
```

Mutante2.java

```
public class Mutante2 {
   public String definirFaixaEtaria(Pessoa p)
                                 throws RuntimeException {
      //O restante igual ao original

      else if(idade < 18)     //'<=' -> '<'

      //O restante igual ao original
   }
}
```

Mutante3.java

```
public class Mutante3 {
   public String definirFaixaEtaria(Pessoa p)
                                 throws RuntimeException {
      //O restante igual ao original

      else if(idade == 59)       //'<=' -> '=='

      //O restante igual ao original
   }
}
```

Implemente casos de teste em JUnit e Mockito para matar cada um dos três mutantes; se algum dos mutantes for equivalente, justifique.

(Exercício 5) Considere as cinco classes a seguir. Observe o programa original que calcula as soma dos fatoriais de números positivos e pares no vetor (lado esquerdo superior) e três mutantes. As mutações aplicadas em cada mutante estão destacadas em negrito e identificadas em comentários.

MathOps.java

```
public interface MathOps {
    public int fatorial(int n);
}
```

Original.java

```
public class Original {
    MathOps mathOps;

    public Original(MathOps mathOps) {
        this.mathOps = mathOps;
    }

    public int somaDeFatoriais(int vet[]) {
        int soma = 0;
        for (int i = 0; i < vet.length; i++) {
            int numero = vet[i];
            if (numero >= 0 && numero % 2 == 0) {
                soma += mathOps.fatorial(numero);
            }
        }
        return soma;
    }
}
```

Mutante1.java

```
public class Mutante1 {
    MathOps mathOps;

    public Mutante1(MathOps mathOps) {
        this.mathOps = mathOps;
    }

    public int somaDeFatoriais(int vet[]) {
        int soma = 0;
        for (int i = 1; i < vet.length; i++) {   // '0' -> '1'
            int numero = vet[i];
            if (numero >= 0 && numero % 2 == 0) {
                soma += mathOps.fatorial(numero);
            }
        }
        return soma;
    }
}
```

Mutante2.java

```
public class Mutante2 {
    MathOps mathOps;

    public Mutante1(MathOps mathOps) {
        this.mathOps = mathOps;
    }

    public int somaDeFatoriais(int vet[]) {
        int soma = 0;
        for (int i = 0; i < vet.length; i++) {
            int numero = vet[i];
            if (numero > 0 && numero % 2 == 0) {    //'>=' -> '>'
                soma += mathOps.fatorial(numero);
            }
        }
        return soma;
    }
}
```

Mutante3.java

```
public class Mutante3 {
    MathOps mathOps;

    public Mutante2(MathOps mathOps) {
        this.mathOps = mathOps;
    }

    public int somaDeFatoriais(int vet[]) {
        int soma = 0;
        for (int i = 0; i < vet.length; i++) {
            int numero = vet[i];
            if (numero >= 0 && numero / 2 == 0) {   // '%' -> '/'
                soma += mathOps.fatorial(numero);
            }
        }
        return soma;
    }
}
```

Implemente casos de teste em JUnit e Mockito para matar cada um dos mutantes; se algum dos mutantes for equivalente, justifique.

(Exercício 6) Considere as quatro classes a seguir. Observe que existe um programa original e três mutantes; as mutações aplicadas em cada mutante estão destacadas em negrito.

Original.java

```
public class Original {
        int[] num = { 1000, 900, 500, 400, 100, 90, 50, 40,
                10, 9, 5, 4, 1 };
        String[] rom = { "M", "CM", "D", "CD", "C", "XC", "L",
                "XL", "X", "IX", "V", "IV", "I" };

        public String coverterParaRomano(int numero) {
                var ret = "";
                var i = 0;
                while (numero > 0) {
                        if (numero >= num[i]) {
                                ret += rom[i];
                                numero -= num[i];
                        } else {
                                i++;
                        }
                }
                return ret;
        }
}
```

Mutante1.java

```
public class Mutante1 {
        int[] num = { 1000, 900, 500, 400, 100, 90, 50, 40,
                10, 9, 5, 4, 1 };
        String[] rom = { "M", "CM", "D", "Cd", "C", "XC", "L",
                "XL", "X", "IX", "V", "IV", "I" };

        public String coverterParaRomano(int numero) {
                var ret = "";
                var i = 0;
                while (numero > 0) {
                        if (numero >= num[i]) {
                                ret += rom[i];
                                numero -= num[i];
                        } else {
                                i++;
                        }
                }
                return ret;
        }
}
```

Mutante2.java

```
public class Mutante2 {
        int[] num = { 10000, 900, 500, 400, 100, 90, 50, 40,
                10, 9, 5, 4, 1 };
        String[] rom = { "M", "CM", "D", "CD", "C", "XC", "L",
                "XL", "X", "IX", "V", "IV", "I" };

        public String coverterParaRomano(int numero) {
                var ret = "";
                var i = 0;
                while (numero > 0) {
                        if (numero >= num[i]) {
                                ret += rom[i];
                                numero -= num[i];
                        } else {
                                i++;
                        }
                }
                return ret;
        }
}
```

Mutante3.java

```
public class Mutante3 {
        int[] num = { 1000, 900, 500, 400, 100, 90, 50, 40,
                10, 9, 5, 4, 1 };
        String[] rom = { "M", "CM", "D", "CD", "C", "XC", "L",
                "XL", "X", "IX", "V", "IV", "I" };

        public String coverterParaRomano(int numero) {
                var ret = "";
                var i = 0;
                while (numero != 0) {
                        if (numero >= num[i]) {
                                ret += rom[i];
                                numero -= num[i];
                        } else {
                                i++;
                        }
                }
                return ret;
        }
}
```

Implemente casos de teste em JUnit para matar cada um dos três mutantes; se algum dos mutantes for equivalente, justifique.